

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**



**BÁO CÁO BÀI TẬP LỚN
MÔN HỌC: HỆ ĐIỀU HÀNH – CO2018**

SIMPLE OPERATING SYSTEM

Nhóm thực hiện:	Nhóm sinh viên lớp L02
Lớp:	L02
Giảng viên hướng dẫn:	Thầy Mai Đức Trung

Thành phố Hồ Chí Minh, tháng 6 năm 2026

DANH SÁCH THÀNH VIÊN THAM GIA

STT	MSSV	Họ và tên	Nhiệm vụ
1	2410155	Trần Nhật Bảo Anh	Hiện thực, kiểm thử và tổng hợp báo cáo
2	2413707	Nguyễn Quốc Trung	Tham gia hiện thực và kiểm thử hệ thống
3	2410297	Trần Hoàng Quốc Bảo	Tham gia hiện thực và phân tích kết quả
4	2410046	Trịnh Quốc An	Tham gia kiểm thử và hoàn thiện tài liệu
5	2410841	Nguyễn Trường Giang	Tham gia kiểm thử và rà soát báo cáo

Bảng 1: Danh sách thành viên nhóm

Mục lục

Danh sách thành viên tham gia	i
Tóm tắt	v
1 Giới thiệu và kiến trúc hệ thống	1
1.1 Mục tiêu bài tập	1
1.2 Đối chiếu trực tiếp với yêu cầu chấm điểm	1
1.3 Kiến trúc tổng quan	2
1.4 Các cấu trúc dữ liệu trung tâm	2
1.5 Định dạng configuration và process description	3
1.6 Vòng đời của một PCB	3
1.7 Phương pháp triển khai và kiểm chứng	4
1.8 Luồng thực thi end-to-end	4
1.9 Cấu hình build đang sử dụng	4
2 CPU Scheduling	5
2.1 Thiết kế Multi-Level Queue	5
2.2 Queue FIFO và vòng đời process	5
2.3 Thuật toán chọn process	5
2.4 Timer và quantum	6
2.5 Phân tích testcase scheduler	6
2.5.1 Ưu tiên và preemption tại biên quantum	6
2.5.2 Hết slot và reset chu kỳ	7
2.5.3 Hai CPU hoạt động song song	7
2.6 Trả lời câu hỏi: lợi ích của các chính sách MLQ	8
2.7 Giới hạn scheduler	8
2.8 Độ phức tạp và đặc tính fairness	9
3 Quản lý bộ nhớ và phân tách User/Kernel	9
3.1 Các tầng của memory subsystem	9
3.2 Process memory layout	9
3.3 Physical memory và free frame list	10
3.4 Page Table Entry	10
3.5 ALLOC và mở rộng VMA	11
3.6 FREE và tái sử dụng region	11
3.7 Bảng chứng trạng thái data segment	11
3.8 READ/WRITE và kiểm tra biên	12
3.9 Không truyền PCB qua syscall	12
3.10 Bảo vệ raw physical access	12
3.11 Kernel allocation và cache pool	13
3.12 Copy giữa user và kernel	13
3.13 Memory swapping và FIFO victim	14
3.14 Các bất biến và đường lỗi của memory subsystem	14
3.15 Trả lời câu hỏi: paging và continuous allocation	15
3.16 Trả lời câu hỏi: kết hợp segmentation và paging	15
4 Multi-Level Paging 64-bit	15

4.1	Sơ đồ địa chỉ 57-bit canonical	15
4.2	Page-table walk	16
4.3	Canonical-address validation	16
4.4	Ví dụ tách index	16
4.5	vmap_pgdl_memset	16
4.6	Lợi ích và chi phí của N-level paging	17
4.7	Hoàn thiện sparse allocation	17
4.8	Phân tích định lượng dung lượng page table	17
5	Synchronization và System Call	18
5.1	Các tài nguyên dùng chung	18
5.2	Ví dụ race condition nếu thiếu lock	18
5.3	Đánh giá phạm vi lock	18
5.4	System-call table và dispatcher	18
5.5	Unified memory syscall	19
5.6	Trả lời câu hỏi: unified syscall interface	19
5.7	Trả lời câu hỏi: syscall chạy quá lâu	19
5.8	Trả lời câu hỏi: hậu quả khi thiếu synchronization	20
6	Kiểm thử và phân tích kết quả	20
6.1	Chiến lược kiểm thử	20
6.2	Traceability từ yêu cầu đến testcase	20
6.3	Kết quả kiểm thử thực tế	21
6.4	Các output thực nghiệm tiêu biểu	22
6.5	Phân tích output MM64	23
6.6	Điểm mạnh của testcase hiện tại	23
6.7	Giới hạn của testcase hiện tại	24
6.8	Đánh giá cơ chế swapping	24
7	Các quyết định thiết kế, giới hạn và hướng cải tiến	24
7.1	Các quyết định thiết kế chính	24
7.2	Giới hạn kỹ thuật đã xác định	25
7.3	Hướng cải tiến	25
7.4	Đối chiếu kết quả với tiêu chí demonstration	25
8	Kết luận	25

Danh sách hình

1	Kiến trúc tổng quát của chương trình mô phỏng	2
2	Vòng đời process trong simulator	3
3	Scheduler quét queue từ priority cao xuống thấp	5
4	Gantt diagram của testcase sched-prio	6
5	Gantt diagram minh họa giới hạn slot và reset chu kỳ	7
6	Gantt diagram của scheduler hai CPU	8
7	Luồng xử lý của memory subsystem	9
8	Mô hình vùng nhớ ảo của một process	10
9	Output thực tế chứng minh vùng data segment được giải phóng và tái sử dụng	11
10	Luồng syscall dùng PID thay vì truyền PCB	12
11	Output thực tế của kiểm tra quyền sở hữu physical frame giữa hai PID	13
12	Output thực tế xác nhận byte 65 được bảo toàn qua user–kernel–user copy	13
13	Chuỗi truy cập của page-table walk năm cấp	16
14	Race condition giả định khi free frame list không được khóa	18
15	Output thực tế của 22 testcase và semantic assertions	22
16	Output thực tế của sparse multi-level paging và canonical-address validation	23

Danh sách bảng

1	Danh sách thành viên nhóm	i
2	Ma trận yêu cầu, implementation và bằng chứng	1
3	Vai trò của các cấu trúc dữ liệu chính	2
4	Instruction set của simulator	3
5	Các mốc quan trọng của sched-prio	7
6	Phân bổ slot trong sched-out-of-slot	7
7	Một số bất biến quan sát được trong sched-2-CPU	8
8	Chi phí của các thao tác scheduler chính	9
9	Các trường quan trọng của PTE	10
10	Bất biến quan trọng của quản lý bộ nhớ	14
11	So sánh paging và continuous allocation	15
12	Phân chia bit của địa chỉ 57-bit	15
13	Dung lượng page table theo các mapping quan sát được	17
14	Tài nguyên dùng chung và cơ chế bảo vệ	18
15	Các syscall trong phiên bản hiện tại	19
16	Liên kết yêu cầu, rủi ro và bằng chứng kiểm thử	20
17	Nhóm testcase hiện có	20
18	Kết quả kiểm chứng tại commit báo cáo	21
19	Giới hạn và tác động	25

Tóm tắt

Bài tập lớn xây dựng một chương trình mô phỏng các thành phần quan trọng của một hệ điều hành đơn giản: bộ lập lịch nhiều mức ưu tiên, đồng bộ hóa nhiều luồng, quản lý bộ nhớ ảo và vật lý, giao diện lời gọi hệ thống, phân tách user/kernel space và bảng trang 64-bit năm cấp. Mỗi CPU được mô phỏng bằng một POSIX thread; loader, timer và các CPU phối hợp qua mutex và condition variable để tạo ra tiến trình thực thi theo từng time slot.

Nhóm triển khai MLQ với 140 hàng đợi và ngân sách slot phụ thuộc priority; chuyển các thao tác memory quan trọng qua syscall dùng PID thay vì truyền trực tiếp PCB; bổ sung kernel allocation, cache pool, kiểm tra quyền truy cập physical memory; và thực hiện page-table walk qua PGD, P4D, PUD, PMD, PT. Báo cáo trình bày thiết kế, đối chiếu mã nguồn, phân tích testcase, thống kê chi phí bảng trang và nêu rõ các giới hạn còn tồn tại.

Phạm vi xác nhận. Các kết luận trong báo cáo dựa trên mã nguồn và các testcase trong thư mục `input/`. Dynamic sparse page-table allocation được kiểm chứng ở branch PMD khác 0; các phép copy user–kernel được kiểm tra bằng giá trị đọc lại. Page replacement end-to-end vẫn được trình bày thận trọng vì chưa có testcase ép RAM cạn rồi xác nhận toàn bộ chu trình swap-out/swap-in.

1 Giới thiệu và kiến trúc hệ thống

1.1 Mục tiêu bài tập

Theo đặc tả, bài tập tập trung vào ba nhóm năng lực chính:

- xây dựng scheduler/dispatcher cho nhiều CPU theo chính sách Multi-Level Queue;
- quản lý memory từ virtual address đến physical frame, đồng thời phân tách user space và kernel space;
- thiết kế giao diện syscall và bảo vệ các tài nguyên dùng chung bằng cơ chế synchronization.

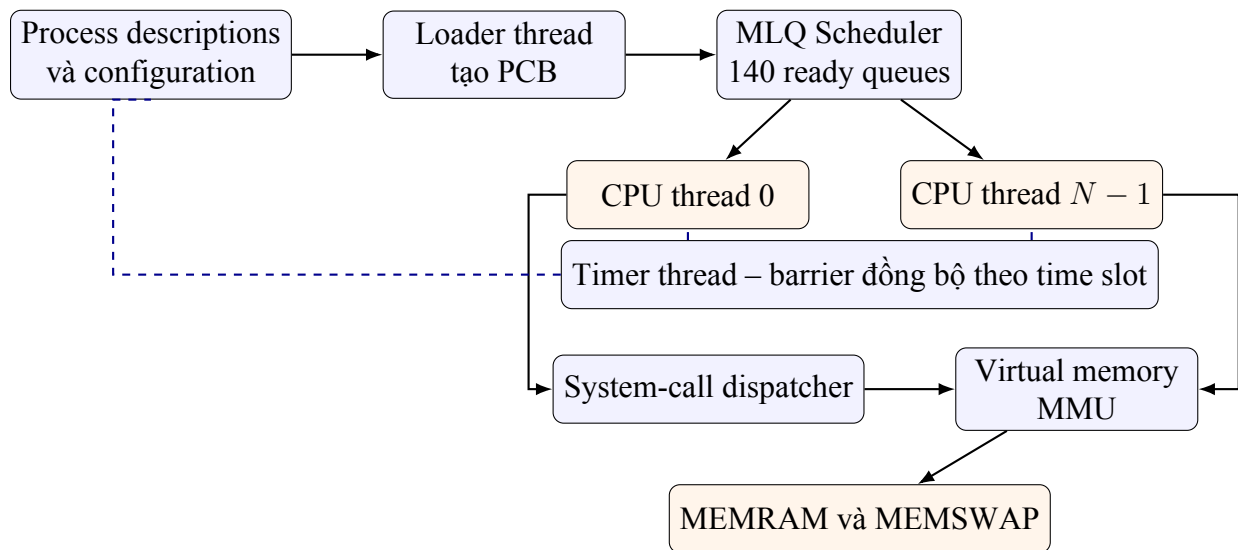
Ngoài ba nhóm trên, phiên bản đề năm 2026 yêu cầu mô phỏng địa chỉ dài 64-bit bằng multi-level paging, cung cấp `vmap_pgd_memset`, thống kê số lần truy cập bảng trang và dung lượng lưu trữ, đồng thời trả lời đầy đủ các câu hỏi lý thuyết.

1.2 Đối chiếu trực tiếp với yêu cầu chấm điểm

Bảng 2: Ma trận yêu cầu, implementation và bằng chứng

Yêu cầu	Implementation chính	Bằng chứng kiểm thử / báo cáo
Scheduling (3 điểm demo)	140 MLQ queues, weighted slot, FIFO, <code>queue_lock</code>	<code>sched_prio</code> , <code>sched_out_of_slot</code> , <code>sched_2_cpu</code> và ba Gantt diagram
MMU và user/kernel (2 điểm demo)	Syscall 17, PID lookup, region/frame ownership, kernel allocation	<code>os_mem_reuse</code> , <code>os_mem_roundtrip</code> , <code>os_cross_pid_protection</code>
Multi-level paging (2 điểm demo)	Canonical 57-bit, PGD–P4D–PUD–PMD–PT, sparse allocation, <code>vmap_pgd_memset</code>	<code>os_mm64_canonical</code> , thống kê walks/accesses/bytes
Questionnaire và report (3 điểm)	Trả lời sáu câu hỏi trong đề; phân tích thiết kế, trade-off và giới hạn	Các mục “Trả lời câu hỏi”, bảng kết quả, hình minh chứng và phân tích giới hạn

1.3 Kiến trúc tổng quan



Hình 1: Kiến trúc tổng quát của chương trình mô phỏng

Luồng thực thi bắt đầu từ file cấu hình. Loader tạo PCB, nạp code segment và đưa PCB vào đúng MLQ queue. Mỗi CPU lấy một process từ scheduler, chạy từng instruction, sau mỗi quantum trả process chưa hoàn thành về queue. Timer đóng vai trò barrier: một time slot chỉ kết thúc khi loader và toàn bộ CPU đang hoạt động đều báo hoàn tất công việc của slot đó.

1.4 Các cấu trúc dữ liệu trung tâm

Bảng 3: Vai trò của các cấu trúc dữ liệu chính

Cấu trúc	Nội dung chính	Vai trò
pcb_t	PID, priority, code, registers, PC, kernel handle, mm	Đại diện trạng thái của một process
krnl_t	scheduler queues, RAM, swap, kernel page tables	Registry dùng chung phía kernel
queue_t	Mảng con trỏ PCB và kích thước	Ready queue và running list
mm_struct	page tables, VMA, region tables, cache pools, thống kê	Address space riêng của process
vm_area_struct	vm_start, vm_end, sbrk, free-region list	Miền địa chỉ ảo liên tục
memphy_struct	storage, size, free frame list	Thiết bị RAM hoặc SWAP mô phỏng

1.5 Định dạng configuration và process description

Chương trình nhận đúng một đối số là tên configuration trong thư mục `input/`. Dòng đầu mô tả time slice, số CPU và số process. Khi `MM_FIXED_MEMSZ` không được bật, dòng thứ hai chỉ định kích thước RAM và tối đa bốn swap device. Các dòng còn lại mô tả thời điểm đến, tên chương trình và priority:

Listing 1: Định dạng file cấu hình

```
1 [time_slice] [number_of_cpus] [number_of_processes]
2 [ram_size] [swap0_size] [swap1_size] [swap2_size] [swap3_size]
3 [arrival_time_0] [program_0] [priority_0]
4 ...
```

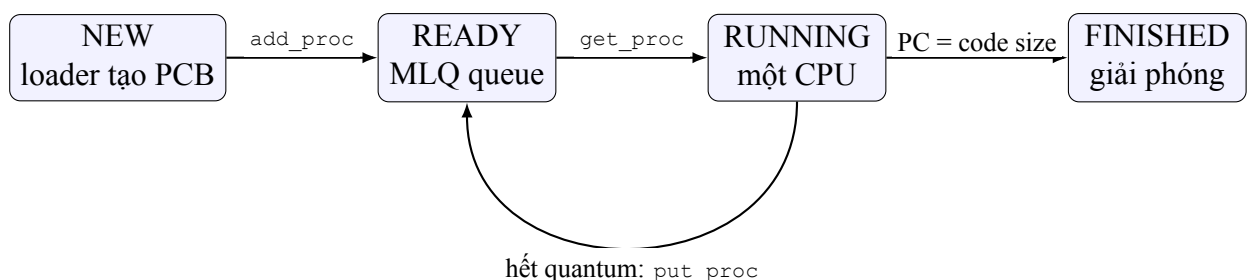
Mỗi file trong `input/proc/` bắt đầu bằng legacy priority và số instruction. Loader đọc lần lượt opcode và argument, chuyển chúng thành mảng `struct inst_t`. Phiên bản hiện tại hỗ trợ các instruction trong Bảng 4.

Bảng 4: Instruction set của simulator

Instruction	Cú pháp rút gọn	Ý nghĩa
CALC	<code>calc</code>	Tiêu thụ một CPU instruction
ALLOC / FREE	<code>alloc size rgid, free rgid</code>	Cấp phát và trả user region
READ / WRITE	<code>read src off dst, write val dst off</code>	Truy cập user memory
KMALLOC	<code>kmalloc size rgid</code>	Cấp kernel region có frame liên tiếp
KMEM CACHE	<code>create / alloc</code>	Tạo pool và cấp object cùng kích thước
COPY	<code>from-user / to-user</code>	Sao chép qua ranh giới user/kernel
SYSCALL	<code>syscall nr a1 a2 a3</code>	Gọi syscall number tương ứng

1.6 Vòng đời của một PCB

PCB được tạo bằng `calloc()`, nhận PID tăng dần từ `avail_pid`, sau đó loader nạp code segment và đặt priority theo configuration. Sau khi được thêm vào scheduler, PCB luân chuyển giữa ready queue và running list cho đến khi program counter bằng code size. CPU lúc đó xóa PCB khỏi running list, in thống kê page table, giải phóng `mm_struct`, code segment, legacy page table và PCB.



Hình 2: Vòng đời process trong simulator

1.7 Phương pháp triển khai và kiểm chứng

Nhóm bắt đầu từ các bất biến mà simulator phải duy trì: một PCB không được chạy đồng thời trên hai CPU, một physical frame không được cấp cho hai process, và userspace không được truyền trực tiếp con trỏ PCB qua ranh giới syscall. Các TODO trong skeleton sau đó được triển khai theo cấu trúc dữ liệu sẵn có. Cuối cùng, mỗi bất biến quan trọng được gắn với ít nhất một testcase hoặc output có thể quan sát.

Quá trình kiểm chứng được chia thành ba mức:

1. **Thành phần:** queue, syscall handler và page-table helper trả kết quả đúng với input cụ thể.
2. **Tích hợp:** loader, scheduler, CPU, timer và memory chạy đồng thời trong cùng binary mà không crash hoặc deadlock quan sát được.
3. **Semantic:** script không chỉ kiểm tra exit code mà còn xác nhận các bất biến đầu ra như cross-PID denial và giá trị round-trip.

1.8 Luồng thực thi end-to-end

Một process description đi qua toàn bộ hệ thống theo chuỗi sau:

1. Loader đọc priority và instruction, khởi tạo PCB cùng address space riêng.
2. Scheduler đặt PCB vào MLQ bucket; CPU lấy PCB dưới `queue_lock`.
3. Với thao tác memory, CPU gọi wrapper; wrapper chuẩn bị scalar register và gọi syscall bằng PID.
4. Kernel tra PID trong scheduler lists, lấy PCB thật rồi kiểm tra region, page table và quyền sở hữu frame.
5. Hết quantum, PCB quay lại ready queue; khi hoàn tất, page table và tài nguyên liên quan được giải phóng.

Luồng này cho thấy scheduler, syscall và memory subsystem phụ thuộc lẫn nhau thay vì là ba module chỉ được kiểm thử riêng rẽ.

1.9 Cấu hình build đang sử dụng

File `include/os-cfg.h` bật đồng thời `MLQ_SCHED`, `MM_PAGING`, `MM64`, `IODUMP` và `PAGETBL_DUMP`. Do đó binary `os` được build với scheduler nhiều mức, paging và địa chỉ 64-bit. Makefile cung cấp:

Listing 2: Các lệnh build và kiểm thử chính

```
1 make clean
2 make all
3 make test
4 make report
```

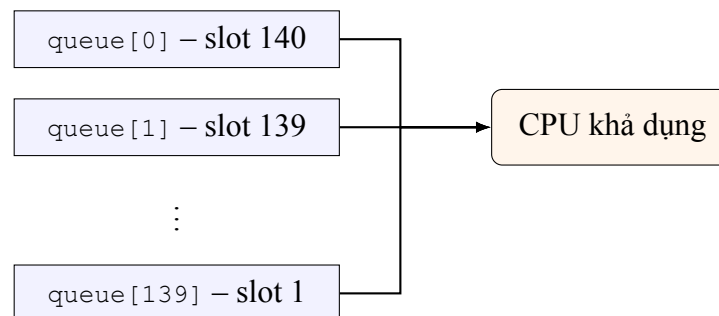
2 CPU Scheduling

2.1 Thiết kế Multi-Level Queue

Scheduler sử dụng `MAX_PRIO` bằng 140; mỗi priority i có một queue riêng trong mảng `mlq_ready_queue`. Giá trị priority càng nhỏ thì mức ưu tiên càng cao. Số lượt scheduler được phép chọn queue i trong một chu kỳ là:

$$slot[i] = MAX_PRIO - i.$$

Vì vậy queue priority 0 có 140 slot, trong khi queue priority 139 chỉ có một slot. Đây là weighted service: process ưu tiên cao nhận nhiều quantum hơn, nhưng process ưu tiên thấp vẫn có cơ hội sau khi các queue phía trên dùng hết ngân sách.



Hình 3: Scheduler quét queue từ priority cao xuống thấp

2.2 Queue FIFO và vòng đời process

`enqueue()` thêm PCB vào cuối queue. `dequeue()` lấy PCB ở index 0 rồi dịch các phần tử còn lại sang trái. Priority không được so sánh bên trong `dequeue()`, vì MLQ đã tách các process theo priority; do đó FIFO trong từng bucket tạo round-robin giữa các process cùng mức.

Listing 3: Logic cốt lõi của queue FIFO

```
1 q->proc[q->size] = proc;
2 q->size++;
3
4 struct pcb_t *ret = q->proc[0];
5 for (i = 0; i < q->size - 1; i++)
6     q->proc[i] = q->proc[i + 1];
7 q->size--;
```

Khi loader gọi `add_proc()`, process được đưa vào queue tương ứng. Khi CPU gọi `get_proc()`, scheduler lấy PCB khỏi ready queue và thêm nó vào `running_list`. Hết quantum, `put_proc()` xóa PCB khỏi `running_list` và trả nó về MLQ queue. Khi process hoàn tất, `finish_proc()` loại PCB khỏi `running_list` trước khi CPU giải phóng tài nguyên.

2.3 Thuật toán chọn process

Listing 4: Pseudo-code của get_mlq_proc()

```

1 lock(queue_lock)
2 for prio = 0 .. MAX_PRIO - 1:
3     if queue[prio] is not empty and slot[prio] > 0:
4         proc = dequeue(queue[prio])
5         slot[prio]--
6         break
7
8 if no process was selected:
9     reset every slot[prio] = MAX_PRIO - prio
10    repeat the scan once
11
12 if proc != NULL:
13     enqueue(running_list, proc)
14 unlock(queue_lock)

```

Toàn bộ thao tác “chọn khỏi ready queue” và “đưa vào running list” nằm trong cùng queue_lock. Vì vậy hai CPU không thể lấy cùng một PCB tại cùng thời điểm.

2.4 Timer và quantum

Mỗi CPU chạy một instruction trong mỗi time slot. Biến time_left được nạp bằng time_slot khi dispatch. Sau mỗi instruction, CPU giảm time_left; khi giá trị về 0, process chưa hoàn tất được đưa lại scheduler. Timer thread chờ toàn bộ event đánh dấu done, tăng global time rồi phát condition signal để bắt đầu slot kế tiếp.

2.5 Phân tích testcase scheduler

2.5.1 Ưu tiên và preemption tại biên quantum

Test sched_prio dùng một CPU, quantum 2 và hai process:

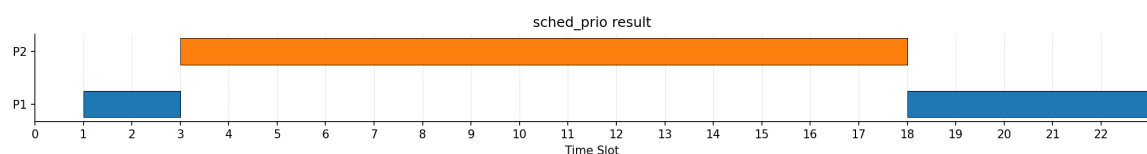
Listing 5: Cấu hình sched_prio

```

1 2 1 2
2 0 s1 1
3 1 s0 0

```

Process priority 1 được nạp trước. Process priority 0 đến sau nhưng không giành CPU giữa quantum đang chạy; nó được chọn ngay tại lần dispatch kế tiếp. Điều này mô phỏng preemption ở biên quantum, không phải preemption giữa một instruction.



Hình 4: Gantt diagram của testcase sched-prio

Bảng 5: Các mốc quan trọng của sched-prio

Time slot	Event	Phân tích
0–1	PID 1, priority 1 được nạp và dispatch	Tại thời điểm đầu chỉ có PID 1 sẵn sàng
1	PID 2, priority 0 được nạp	PID 1 vẫn hoàn thành quantum hiện tại
3	PID 1 được put; PID 2 dispatch	Scheduler chọn priority 0 tại biên quantum
3–18	PID 2 tiếp tục được chọn	Queue priority 0 còn slot và có mức ưu tiên cao nhất
18	PID 2 hoàn thành; PID 1 quay lại	Process thấp hơn không bị mất, chỉ phải chờ

2.5.2 Hết slot và reset chu kỳ

`sched_out_of_slot` tạo các priority 139, 138 và 137, tương ứng ngân sách 1, 2 và 3 slot. Khi queue ưu tiên cao đã dùng hết slot, scheduler chuyển xuống queue kế tiếp dù process ưu tiên cao vẫn chưa hoàn thành. Sau khi tất cả queue có process đều không còn slot khả dụng, toàn bộ ngân sách được reset.



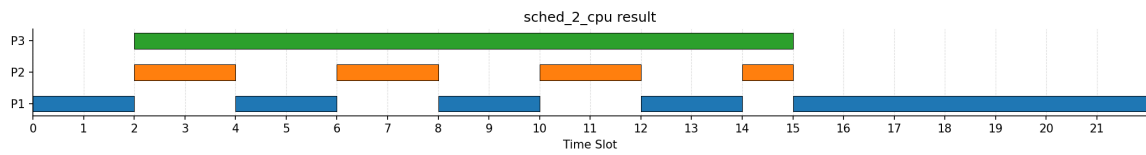
Hình 5: Gantt diagram minh họa giới hạn slot và reset chu kỳ

Bảng 6: Phân bổ slot trong sched-out-of-slot

Priority	Ngân sách	Khoảng chạy đầu tiên	Lý do chuyển queue
139	1	slot 1–3	Dùng hết một slot
137	3	slot 3–9	Dùng hết ba slot
138	2	slot 9–13	Dùng hết hai slot
137	reset	từ slot 13	Tất cả queue khả dụng đã hết ngân sách

2.5.3 Hai CPU hoạt động song song

`sched_2_cpu` có hai CPU và ba process. Trong một thời điểm, tối đa hai PCB khác nhau xuất hiện trên hai CPU. Thứ tự log giữa các thread có thể thay đổi qua từng lần chạy, nhưng các bất biến phải giữ nguyên: một PCB không chạy trên hai CPU cùng lúc, mỗi process tuân thủ quantum, và process mới được chọn từ MLQ dưới lock.



Hình 6: Gantt diagram của scheduler hai CPU

Bảng 7: Một số bất biến quan sát được trong sched-2-CPU

Bất biến	Bằng chứng từ log
Không dispatch cùng PID cho hai CPU	Mỗi thời điểm CPU 0 và CPU 1 nhận PID khác nhau
Process ưu tiên cao được chọn sớm	PID 3 priority 137 được CPU 1 lấy ngay sau khi PID 1 hết quantum
Quantum được tôn trọng	Các dòng “Put process” xuất hiện ở biên hai time slot
CPU tiếp tục khi CPU khác rảnh/dừng	Sau khi PID 2 và PID 3 hoàn tất, CPU 0 tiếp tục PID 1

2.6 Trả lời câu hỏi: lợi ích của các chính sách MLQ

- **Priority queue:** giảm response time cho workload quan trọng bằng cách quét từ priority nhỏ đến lớn.
- **Weighted slot:** biến priority thành tỷ lệ CPU time thay vì cấm hoàn toàn process thấp hơn.
- **FIFO trong cùng queue:** tạo fairness giữa các process có cùng priority.
- **Reset slot:** bảo đảm scheduler tiếp tục hoạt động khi các queue đang có process đều đã hết ngân sách.
- **Time slice:** ngăn một process giữ CPU vô thời hạn và tạo điểm chuyển context có kiểm soát.

2.7 Giới hạn scheduler

MLQ hiện tại không có feedback để thay đổi priority theo hành vi process. Khi một queue priority rất cao có nhiều process, process priority thấp vẫn có thể phải chờ lâu. Ngoài ra, `queue_empty()` đọc queue mà không lấy `queue_lock`; hàm này hiện không được dùng trong luồng chính, nhưng sẽ cần bảo vệ nếu được tái sử dụng.

2.8 Độ phức tạp và đặc tính fairness

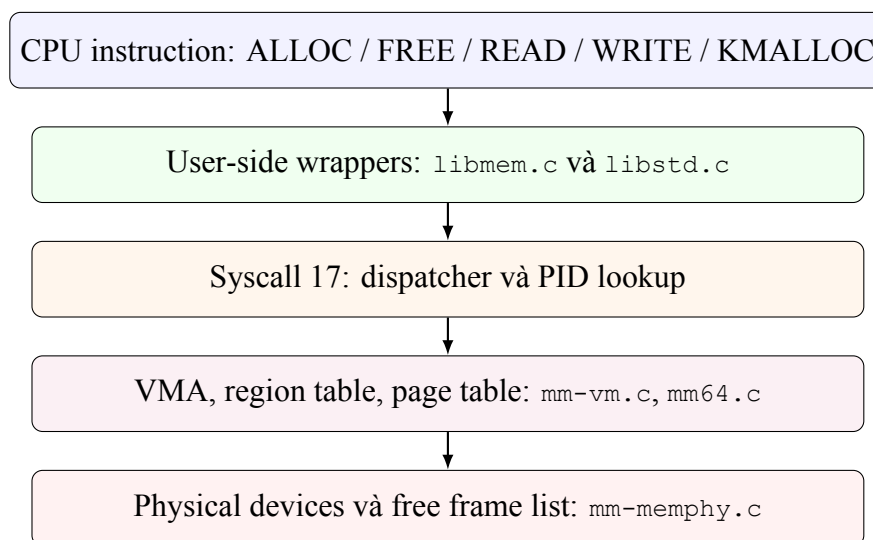
Bảng 8: Chi phí của các thao tác scheduler chính

Thao tác	Độ phức tạp	Nhận xét
<code>enqueue()</code>	$O(1)$	Ghi PCB vào cuối mảng queue
<code>dequeue()</code>	$O(n)$	Dịch các phần tử còn lại sang trái
Chọn MLQ process	$O(140 + n)$	Quét tối đa 140 queue rồi dequeue bucket được chọn
Tra cứu PID	$O(P + 140)$	Duyệt process trong các queue dưới lock

Với quy mô bài tập, mảng queue giúp implementation trực quan và dễ kiểm chứng. Trong hệ thống lớn hơn, circular buffer sẽ giảm `dequeue()` xuống $O(1)$, còn PID lookup nên dùng hash table. Weighted slot giúp priority thấp vẫn nhận CPU sau khi các bucket đang hoạt động dùng hết ngân sách, nhưng không đưa ra upper bound chặt cho waiting time nếu process ưu tiên cao liên tục xuất hiện.

3 Quản lý bộ nhớ và phân tách User/Kernel

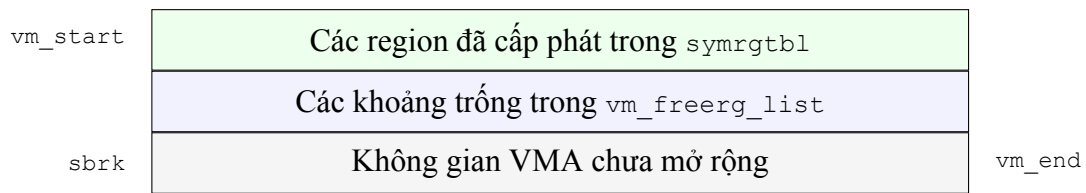
3.1 Các tầng của memory subsystem



Hình 7: Luồng xử lý của memory subsystem

3.2 Process memory layout

Mỗi process sở hữu một `mm_struct`. Trong đó `mmap` trỏ đến danh sách VMA; phiên bản hiện tại tập trung vào VMA 0. Một VMA mô tả miền liên tục $[vm_start, vm_end)$, còn `sbrk` đánh dấu biên đã sử dụng. Các allocation được nhận diện bằng integer region ID và lưu trong `symrgtbl[rgid]`; cách tra cứu này có độ phức tạp $O(1)$ và giới hạn tối đa 30 region trên mỗi process.



Hình 8: Mô hình vùng nhớ ảo của một process

3.3 Physical memory và free frame list

`memphy_struct` được dùng cho cả RAM và SWAP. `storage` là mảng byte chứa dữ liệu; `maxsz` là dung lượng thiết bị; `free_fp_list` giữ các frame chưa được sử dụng. Khi format thiết bị, mỗi frame number được đưa vào linked list. Hai thao tác quan trọng là:

- `MEMPHY_get_freefp()`: lấy head, cập nhật list và giải phóng node;
- `MEMPHY_put_freefp()`: tạo node mới và đưa frame về đầu list.

Hai hàm đều dùng `memphy_lock`. Hàm `MEMPHY_get_contiguous_fps()` tìm đủ một dãy frame number liên tiếp cho kernel allocation. Việc tìm kiếm hiện có độ phức tạp cao vì với mỗi candidate, hàm duyệt lại toàn bộ list cho từng offset; đây là đánh đổi chấp nhận được trong simulator nhỏ.

Hai granularity cùng tồn tại. Framework cấp phát user/kernel memory vẫn dùng frame vật lý 256 byte theo `PAGING_PAGESZ`. Riêng yêu cầu MM64 dùng địa chỉ canonical 57-bit với offset 12 bit và bảng 512 entry, tức page-table granularity 4 KiB. Test dummy mapping đánh giá cấu trúc MM64; các test ALLOC/READ/WRITE đánh giá framework memory hiện hữu. Báo cáo tách rõ hai phạm vi này để không đồng nhất sai hai khái niệm.

3.4 Page Table Entry

Dù page-table hierarchy sử dụng pointer 64-bit, leaf PTE vẫn tận dụng layout trạng thái 32-bit của framework gốc. Các bit quan trọng được trình bày trong Bảng 9.

Bảng 9: Các trường quan trọng của PTE

Trường	Bit	Ý nghĩa
FPN	0–12	Frame number khi page đang ở RAM
SWPTYP	0–4	Loại swap device khi page được swap
SWPOFF	5–25	Frame/offset trên swap device
DIRTY	28	Page đã bị thay đổi
SWAPPED	30	PTE mô tả page trên swap
PRESENT	31	PTE hợp lệ/đã được thiết lập theo framework

Các macro `SETBIT`, `CLRBIT`, `SETVAL` và `GETVAL` thao tác trực tiếp trên bitmask. Cách biểu diễn giúp PTE gọn, nhưng cần thống nhất chặt chẽ ý nghĩa của tổ hợp `PRESENT/SWAPPED`; nếu không, page fault logic dễ diễn giải sai trạng thái.

3.5 ALLOC và mở rộng VMA

`__alloc()` trước hết gọi `get_free_vmrg_area()` để tái sử dụng vùng trống. Nếu không có vùng đủ lớn, `inc_vma_limit()` căn chỉnh kích thước lên page boundary, lấy các frame từ RAM, map chúng vào page table, cập nhật `vm_end` và `sbrk`, sau đó trả virtual address về wrapper thông qua `sc_regs`.

1. Xác thực `rgid`, kích thước và VMA.
2. Tìm free region phù hợp.
3. Nếu thất bại, tính số page cần thêm và lấy frame từ `free_fp_list`.
4. Ghi PTE cho từng page và cập nhật region table.
5. Ghi địa chỉ trả về vào register của process.

3.6 FREE và tái sử dụng region

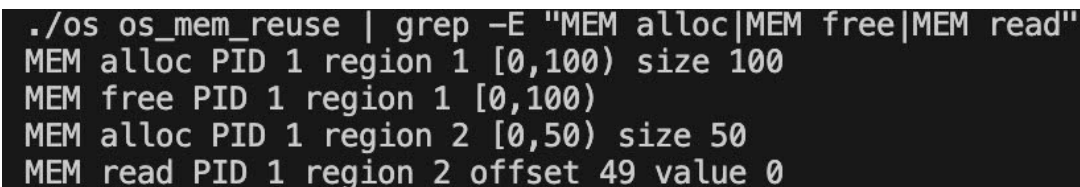
`__free()` không lập tức thu hồi các physical frame của user allocation. Nó xóa entry tương ứng trong symbol region table và đưa vùng virtual address trở lại `vm_freerg_list`. Allocation sau có thể cắt một phần vùng này để dùng lại. Thiết kế làm giảm số lần mở rộng VMA, nhưng free-region list chưa thực hiện coalescing các vùng kề nhau nên có thể xuất hiện fragmentation trong danh sách.

3.7 Bảng chứng trạng thái data segment

Đề yêu cầu báo cáo thể hiện trạng thái cấp phát trong data segment. Test `os_mem_reuse` ghi trực tiếp ranh giới region trước và sau `FREE`:

Listing 6: Cấp phát, giải phóng và tái sử dụng data region

```
MEM alloc PID 1 region 1 [0,100) size 100
MEM free PID 1 region 1 [0,100)
MEM alloc PID 1 region 2 [0,50) size 50
MEM read PID 1 region 2 offset 49 value 0
```



```
./os os_mem_reuse | grep -E "MEM alloc|MEM free|MEM read"
MEM alloc PID 1 region 1 [0,100) size 100
MEM free PID 1 region 1 [0,100)
MEM alloc PID 1 region 2 [0,50) size 50
MEM read PID 1 region 2 offset 49 value 0
```

Hình 9: Output thực tế chứng minh vùng data segment được giải phóng và tái sử dụng

Region 2 bắt đầu lại tại địa chỉ 0, chứng minh allocator cắt và tái sử dụng vùng `[0,100)` vừa được trả về free-region list. Phép đọc offset 49 thành công xác nhận biên cuối hợp lệ của region mới là địa chỉ 49; offset từ 50 trở đi phải bị từ chối.

3.8 READ/WRITE và kiểm tra biên

`__read()` và `__write()` xác định region từ `rgid`, từ chối region rỗng hoặc offset nằm ngoài `[rg_start, rg_end)`. Virtual address được tách thành page number và offset; PTE cung cấp frame number; physical address cuối cùng là:

$$physical_address = fpn \times PAGING_PAGESZ + offset.$$

Thao tác physical read/write tiếp tục đi qua syscall để kernel kiểm tra quyền sở hữu frame.

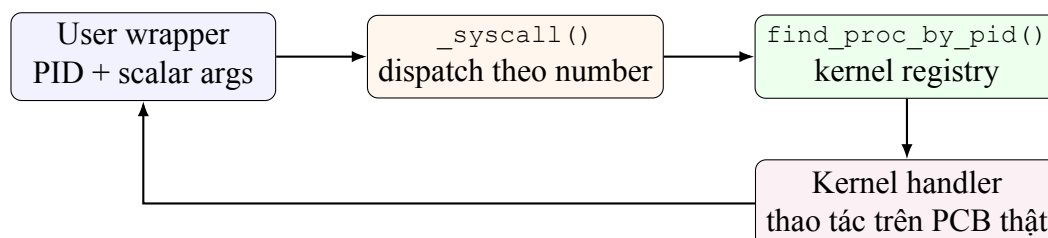
3.9 Không truyền PCB qua syscall

Đặc tả cấm userspace truyền trực tiếp con trỏ PCB. Trong implementation, wrapper chỉ dùng PCB cục bộ để lấy hai giá trị: kernel handle và PID. Hàm dispatcher nhận `krnl`, `pid`, syscall number và scalar registers:

Listing 7: Giao diện syscall không truyền PCB

```
1 int _syscall(struct krnl_t *krnl, uint32_t pid,
2             uint32_t nr, struct sc_regs *regs);
3
4 return _syscall(proc->krnl, proc->pid, 17, &regs);
```

Kernel gọi `find_proc_by_pid()` để duyệt `running_list`, `ready_queue`, `run_queue` và toàn bộ MLQ queues dưới `queue_lock`. PCB thực tế chỉ được lấy từ registry kernel, ngăn caller cung cấp một con trỏ PCB giả.



Hình 10: Luồng syscall dùng PID thay vì truyền PCB

3.10 Bảo vệ raw physical access

`caller_owns_phyaddr()` duyệt page table của caller để xác nhận frame chứa physical address đang được map và không ở trạng thái swapped. Nếu không, syscall từ chối truy cập. Test `os_mem_protection` cố đọc/ghi physical address 0 và nhận:

Listing 8: Output của kiểm tra physical memory protection

```
Kernel denied PID 1 physical read at 0x0
Kernel denied PID 1 physical write at 0x0
```

Test mạnh hơn `os_cross_pid_protection` cho PID 1 cấp region nằm trên frame vật lý 0, sau đó PID 2 cố đọc và ghi đúng frame đang tồn tại. Kernel vẫn từ chối:

Listing 9: Protection giữa hai PID

```
MEM alloc PID 1 region 1 [0,100) size 100
Kernel denied PID 2 physical read at 0x0
Kernel denied PID 2 physical write at 0x0
```

```
./os os_cross_pid_protection | grep -E "MEM alloc|Kernel denied"
MEM alloc PID 1 region 1 [0,100) size 100
Kernel denied PID 2 physical read at 0x0
Kernel denied PID 2 physical write at 0x0
```

Hình 11: Output thực tế của kiểm tra quyền sở hữu physical frame giữa hai PID

Do đó protection không chỉ từ chối địa chỉ chưa map, mà còn kiểm tra ownership theo PID.

3.11 Kernel allocation và cache pool

`__kmalloc()` cấp một dãy physical frame liên tiếp bằng helper tìm contiguous frames, sau đó gán virtual address thuộc miền `PAGING_KMEM_BASE`. Kernel region được lưu trong `ksymrgtbl`, tách khỏi `symrgtbl` của userspace.

Cache pool dùng một kernel allocation lớn làm backing storage. Với object size `align`, lần cấp phát thứ i trả:

$$object_address = pool.storage + i \times pool.align.$$

Đây là mô hình slab/cache đơn giản: object có cùng kích thước được cấp tuần tự từ pool. Implementation không phải buddy allocator đầy đủ vì không chia và gộp block theo lũy thừa hai; thay vào đó nó tìm trực tiếp một chuỗi frame liên tiếp.

3.12 Copy giữa user và kernel

`copy_from_user` và `copy_to_user` sao chép từng byte qua các helper có kiểm tra region. Kernel không dereference trực tiếp một user pointer tùy ý. Cách làm này an toàn hơn về ownership, dù chi phí cao do lặp từng byte và nhiều lần page-table walk.

```
./os os_mem_roundtrip | grep -E "MEM alloc|MEM read"
MEM alloc PID 1 region 1 [0,16) size 16
MEM alloc PID 1 region 3 [16,32) size 16
MEM read PID 1 region 3 offset 0 value 65
```

Hình 12: Output thực tế xác nhận byte 65 được bảo toàn qua user–kernel–user copy

Hai allocation tạo region nguồn `[0, 16)` và region đích `[16, 32)`. Giá trị 65 đọc lại từ region 3 cho thấy chuỗi copy không làm thay đổi dữ liệu và region đích được kernel truy cập thông qua giao diện có kiểm tra.

3.13 Memory swapping và FIFO victim

`fifo_pgn` lưu các page number đã được map. `find_victim_page()` đi đến cuối linked list, lấy page cũ nhất và xóa node đó. Khi page đích không present, `pg_getpage()` dự kiến thực hiện chuỗi:

1. chọn victim page và lấy một frame trên swap;
2. copy victim từ RAM sang swap, cập nhật victim PTE;
3. dùng frame RAM vừa giải phóng cho page đích;
4. nếu page đích từng nằm trên swap, copy dữ liệu trở lại RAM;
5. cập nhật PTE và đưa page đích vào FIFO list.

Ý tưởng này phản ánh page replacement FIFO. Implementation tách riêng operation swap-out (RAM sang swap) và swap-in (swap sang RAM), đồng thời clear bit PRESENT khi PTE chuyển sang trạng thái swapped. Mức độ kiểm chứng được phân tích tại Mục 6.8.

3.14 Các bất biến và đường lỗi của memory subsystem

Bảng 10: Bất biến quan trọng của quản lý bộ nhớ

Bất biến	Cơ chế duy trì	Bảng chứng
Region hợp lệ nằm trong VMA	Kiểm tra <code>rgid</code> , ranh giới và overlap	<code>os_mem_reuse</code>
Frame chỉ thuộc process được map	Duyệt page table trước raw physical I/O	<code>os_cross_pid_protection</code>
User pointer không được kernel dereference trực tiếp	Copy qua helper có kiểm tra region	<code>os_mem_roundtrip</code>
PTE swapped không đồng thời PRESENT	Helper đổi trạng thái clear bit PRESENT	Phân tích source và trạng thái PTE
Page-table tree được thu hồi khi process kết thúc	Duyệt và giải phóng đệ quy	Các test tích hợp kết thúc không crash

Memory API trả lỗi sớm khi region ID không hợp lệ, offset vượt biên, địa chỉ không canonical hoặc caller không sở hữu physical frame. Việc kiểm tra trước thao tác ghi tránh trạng thái cập nhật một phần. Khi không lấy đủ frame, allocation hiện trả lỗi thay vì tạo mapping thiếu; hành vi này an toàn dù chưa tận dụng page replacement để tiếp tục cấp phát.

3.15 Trả lời câu hỏi: paging và continuous allocation

Bảng 11: So sánh paging và continuous allocation

Tiêu chí	Paging	Continuous allocation
Fragmentation	Loại bỏ external fragmentation, có internal fragmentation ở page cuối	Dễ external fragmentation
Dịch địa chỉ	Cần page table và nhiều memory access	Đơn giản với base/limit
Cấp vùng lớn	Không cần frame liên tiếp	Phải tìm block liên tiếp
Protection	Có thể đặt quyền theo page	Thường đặt quyền theo cả segment/block
Swapping	Có thể swap từng page	Thường phải di chuyển vùng lớn

3.16 Trả lời câu hỏi: kết hợp segmentation và paging

Segmentation biểu diễn cấu trúc logic như code, data, heap và stack; paging cung cấp đơn vị quản lý vật lý cố định. Kết hợp hai cơ chế cho phép đặt protection theo segment nhưng map mỗi segment bằng các frame rời rạc, nhờ đó tránh yêu cầu một block physical liên tiếp và giảm external fragmentation. Trong simulator, VMA/region đóng vai trò gần với segmentation, còn page table thực hiện ánh xạ xuống frame.

4 Multi-Level Paging 64-bit

4.1 Sơ đồ địa chỉ 57-bit canonical

Chế độ MM64 dùng kiểu `addr_t = uint64_t`. Địa chỉ canonical 57-bit yêu cầu bit 63–57 là sign extension của bit 56. Năm index, mỗi index 9 bit, và offset 12 bit tạo thành:

$$\underbrace{PGD}_9 \underbrace{P4D}_9 \underbrace{PUD}_9 \underbrace{PMD}_9 \underbrace{PT}_9 \underbrace{Offset}_{12}.$$

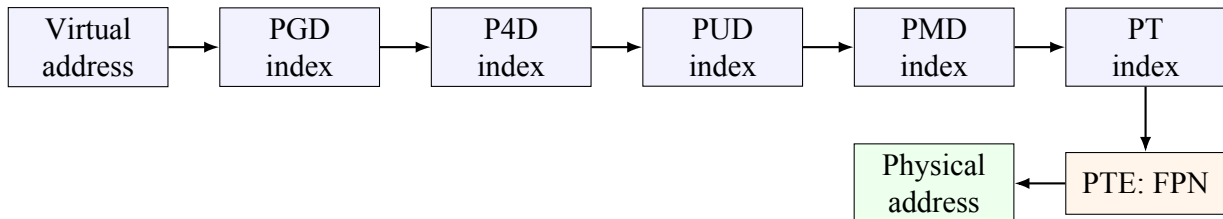
Bảng 12: Phân chia bit của địa chỉ 57-bit

Cấp	Dải bit	Số entry	Dung lượng một bảng
PGD	56–48	512	4096 byte
P4D	47–39	512	4096 byte
PUD	38–30	512	4096 byte
PMD	29–21	512	4096 byte
PT	20–12	512	4096 byte
Offset	11–0	4096 vị trí	–

4.2 Page-table walk

`walk_pte()` lần lượt đọc entry tại PGD, P4D, PUD, PMD rồi trả con trỏ tới PTE. Mỗi walk hợp lệ tăng `pgtable_walks` một lần và tăng `pgtable_accesses` năm lần. PGD được cấp lúc khởi tạo; các bảng con được cấp động khi walk tạo mapping. Với nhánh địa chỉ đầu tiên:

$$pgtable_bytes = 5 \times 512 \times 8 = 20\,480 \text{ byte.}$$



Hình 13: Chuỗi truy cập của page-table walk năm cấp

4.3 Canonical-address validation

`paging64_is_canonical()` lấy phần upper bằng `addr >> 57` và bit sign bằng `(addr >> 56) & 1`. Nếu sign bằng 0 thì upper phải bằng 0; nếu sign bằng 1 thì upper phải bằng `0x7f`. Test gửi địa chỉ `0x0100000000000000`, có bit cao không phải sign extension, và kernel trả:

Listing 10: Kết quả test địa chỉ non-canonical

```
MM64 rejected non-canonical address 0x1000000000000000
```

4.4 Ví dụ tách index

Với địa chỉ canonical thấp `0x000000000001ab123`, các cấp trên đều có index 0, PT index bằng `0x1ab` và offset bằng `0x123`. Công thức tổng quát:

$$\begin{aligned} pgd &= (addr \gg 48) \& 0x1ff, & p4d &= (addr \gg 39) \& 0x1ff, \\ pud &= (addr \gg 30) \& 0x1ff, & pmd &= (addr \gg 21) \& 0x1ff, \\ pt &= (addr \gg 12) \& 0x1ff, & offset &= addr \& 0xfff. \end{aligned}$$

Test tại `0x200000` cho index PMD bằng 1, chứng minh implementation không bị giới hạn ở branch index 0.

4.5 `vmap_pgdstemset`

Theo đề, hàm này dùng để mô phỏng việc chạm vào page-directory trong không gian địa chỉ lớn mà không cần cấp frame vật lý thật. Implementation hiện tại kiểm tra canonical address, alignment và range, sau đó cấp các bảng con còn thiếu, walk đến từng PTE và đặt giá trị 0. Vì vậy testcase chứng minh được canonical validation lẫn sparse allocation ở địa chỉ xa.

4.6 Lợi ích và chi phí của N-level paging

Nếu dùng flat page table cho toàn bộ không gian 57-bit với page 4 KiB, số page lý thuyết là 2^{45} . Với mỗi PTE 8 byte, flat table cần:

$$2^{45} \times 8 = 2^{48} \text{ byte} = 256 \text{ TiB.}$$

Hierarchical paging cho phép chỉ tạo các bảng con cho vùng đang dùng. Đổi lại, một TLB miss có thể cần năm lần đọc bảng trước khi truy cập dữ liệu thật. TLB và caching là các cơ chế thường dùng để giảm chi phí này.

4.7 Hoàn thiện sparse allocation

`init_mm()` chỉ cấp PGD ban đầu. Hàm `walk_pte()` nhận cờ tạo mới và dùng `calloc()` để cấp P4D, PUD, PMD hoặc PT còn thiếu. Khi process kết thúc, cây được duyệt đệ quy để hoàn trả frame user/swap rồi giải phóng mọi bảng con. Do đó:

- địa chỉ thấp đầu tiên dùng 20 480 byte cho năm bảng;
- mapping tại `0x200000` tạo thêm một PT, nâng tổng lên 24 576 byte;
- ownership check và cleanup duyệt cây động thay vì giả định mảng PT nhánh 0.

4.8 Phân tích định lượng dung lượng page table

Sparse allocation làm dung lượng phụ thuộc số branch thực sự được chạm thay vì kích thước toàn bộ virtual address space:

Bảng 13: Dung lượng page table theo các mapping quan sát được

Trạng thái	Dung lượng	Giải thích
Chỉ có PGD sau <code>init_mm()</code>	4096 byte	Chưa có bảng con
Mapping đầu tiên tại <code>0x0</code>	20 480 byte	PGD, P4D, PUD, PMD và PT
Thêm mapping tại <code>0x200000</code>	24 576 byte	Dùng chung cấp trên, tạo thêm một PT
Thêm high-canonical branch	40 960 byte	Tạo đường đi mới từ PGD index 256

Một mapping xa không buộc hệ thống cấp toàn bộ entry trung gian cho vùng chưa sử dụng. Kết quả tăng theo từng branch là bằng chứng định lượng cho lợi ích của hierarchical paging đối với không gian địa chỉ dài.

5 Synchronization và System Call

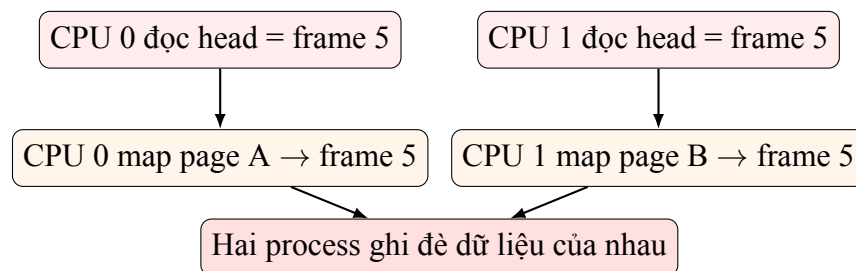
5.1 Các tài nguyên dùng chung

Bảng 14: Tài nguyên dùng chung và cơ chế bảo vệ

Tài nguyên	Lock / primitive	Mục đích
MLQ queues, run-ning list	queue_lock	Tránh hai CPU lấy cùng PCB; bảo vệ PID lookup
Free physical frame list	memphy_lock	Tránh cấp hoặc free cùng frame đồng thời
VMA, region table, kernel cache	mmvm_lock	Tuần tự hóa alloc/free và cập nhật metadata
Timer events	mutex + condition variable	Barrier giữa timer, loader và CPU
Loader completion	atomic_int done	Báo CPU dừng khi không còn process mới

5.2 Ví dụ race condition nếu thiếu lock

Nếu hai CPU cùng đọc head của `free_fp_list` trước khi một CPU cập nhật head, cả hai có thể nhận cùng một frame. Hai page table khác nhau sau đó cùng map đến frame đó, gây rò rỉ dữ liệu và corruption. Tương tự, nếu dequeue không atomic, hai CPU có thể cùng nhận một PCB và tăng program counter song song.



Hình 14: Race condition giả định khi free frame list không được khóa

5.3 Đánh giá phạm vi lock

Thiết kế dùng các global mutex đơn giản và dễ chứng minh tính đúng đắn. Đổi lại, mọi process đều bị tuần tự hóa khi thao tác memory metadata, kể cả khi chúng có address space riêng. Một cải tiến hợp lý là dùng lock theo `mm_struct` kết hợp lock riêng cho physical allocator, giúp tăng parallelism mà vẫn giữ ownership rõ ràng.

5.4 System-call table và dispatcher

File `syscall.tbl` định nghĩa ba syscall:

Bảng 15: Các syscall trong phiên bản hiện tại

Number	Tên	Chức năng
0	<code>listsyscall</code>	In toàn bộ syscall table
17	<code>memmap</code>	Dispatcher cho các memory operation
440	<code>xxx</code>	Syscall minh họa nhận tham số scalar

Macro `__SYSCALL` được dùng hai lần: một lần sinh declaration cho handler, một lần sinh các case trong switch dispatcher. Cách này giữ syscall number và handler mapping trong một nguồn dữ liệu duy nhất.

5.5 Unified memory syscall

Syscall 17 dùng `regs->a1` làm operation code. Các operation gồm map, tăng VMA, swap copy, physical I/O, alloc/free/read/write, kmalloc, cache create/alloc và copy giữa user/kernel. Ưu điểm là tất cả memory operation đi qua một điểm kiểm tra PID và quyền; nhược điểm là handler có switch lớn và operation-specific argument convention.

5.6 Trả lời câu hỏi: unified syscall interface

Ưu điểm:

- tạo abstraction ổn định giữa chương trình user và implementation kernel;
- tập trung validation, protection và error handling;
- giảm coupling và giúp thay đổi implementation mà không đổi user API;
- tăng portability khi các tài nguyên dùng cùng mô hình read/write/free.

Nhược điểm:

- trap, dispatch và validation tạo overhead;
- một giao diện quá tổng quát có thể che mất tính năng chuyên biệt;
- error code tổng quát khó biểu diễn đủ ngữ cảnh;
- switch dispatcher lớn làm tăng độ phức tạp bảo trì.

5.7 Trả lời câu hỏi: syscall chạy quá lâu

Hệ điều hành thực tế dùng timer interrupt hoặc watchdog để phát hiện tác vụ giữ CPU quá lâu. Với preemptible kernel, scheduler có thể tạm dừng tác vụ tại điểm an toàn. Nếu syscall đang chờ I/O, process nên chuyển sang blocked state để CPU phục vụ process khác. Nếu tác vụ vi phạm timeout hoặc mắc deadlock không thể phục hồi, kernel có thể trả lỗi, gửi signal, kết thúc process hoặc kích hoạt watchdog recovery. Kernel code cũng phải tránh giữ lock trong thao tác dài vì điều đó có thể làm toàn hệ thống mất tiến triển.

5.8 Trả lời câu hỏi: hậu quả khi thiếu synchronization

Thiếu synchronization có thể gây lost update, duplicate allocation, double-free, PCB chạy trên nhiều CPU, page table bị ghi đè và output không tắt định theo cách không giải thích được. Trong simulator, hai ví dụ trực tiếp nhất là cùng dequeue một PCB và cùng lấy head của free frame list. Lock không làm thứ tự log hoàn toàn cố định, nhưng bảo vệ các bất biến dữ liệu.

6 Kiểm thử và phân tích kết quả

6.1 Chiến lược kiểm thử

Script `run.sh` chạy 22 cấu hình, bao phủ scheduler, paging, syscall, canonical address, data-region reuse, cross-PID protection và copy user–kernel. Sau khi chạy, script `tests/assert\_outputs.sh` kiểm tra các bất biến semantic thay vì so khớp toàn bộ log đa luồng. Ba testcase scheduler tự tạo còn được chuyển thành Gantt diagram. Build sạch bằng `make clean \&\& make all` hoàn tất với `-Wall` mà không warning.

6.2 Traceability từ yêu cầu đến testcase

Bảng 16: Liên kết yêu cầu, rủi ro và bằng chứng kiểm thử

Yêu cầu	Rủi ro cần phát hiện	Test/bằng chứng chính
MLQ nhiều CPU	Hai CPU lấy cùng PCB; priority hoặc slot sai	<code>sched_prio</code> , <code>sched_out_of_slot</code> , <code>sched_2_cpu</code>
Data-segment allocation	Không tái sử dụng vùng free; sai ranh giới	<code>os_mem_reuse</code> , Hình 9
User/kernel separation	Caller đọc frame thuộc PID khác	<code>os_cross_pid_protection</code> , Hình 11
Copy user/kernel	Dữ liệu thay đổi hoặc truy cập sai region	<code>os_mem_roundtrip</code> , Hình 12
Multi-level paging	Chỉ chạy branch 0; nhận địa chỉ non-canonical	<code>os_mm64_canonical</code> , Hình 16

Bảng 17: Nhóm testcase hiện có

Test	Thành phần	Mục tiêu
<code>sched</code> , <code>sched_0</code> , <code>sched_1</code>	Scheduler	MLQ cơ bản và multi-CPU
<code>sched_prio</code>	Scheduler	Process ưu tiên cao đến sau
<code>sched_out_of_slot</code>	Scheduler	Giới hạn slot và reset
<code>sched_2_cpu</code>	Scheduler	Hai CPU lấy PCB khác nhau
<code>os_0/1/2_mlq_paging</code>	Integration	Scheduler kết hợp paging
<code>os_1_mlq_paging_small_1K</code>	Memory	RAM nhỏ và thiếu frame

Test	Thành phần	Mục tiêu
os_1_mfq_paging_small_4K os_syscall, os_syscall_list os_mm64_canonical	Memory Syscall MM64	RAM nhỏ và thiếu frame Dispatch và handler
os_mem_reuse	Memory	Cấp branch PMD khác 0; từ chối non-canonical
os_mem_roundtrip	Memory	Cấp phát, free và tái sử dụng data region
os_mem_protection	Protection	Copy user–kernel–user giữ nguyên byte
os_cross_pid_protection	Protection	Từ chối raw physical access
		Từ chối truy cập frame thuộc PID khác

6.3 Kết quả kiểm thử thực tế

Bảng 18: Kết quả kiểm chứng tại commit báo cáo

Lệnh / kịch bản	Kết quả	Ghi chú
make clean && make all	PASS	Build sạch, không warning từ -Wall
make test	PASS	22 kịch bản và toàn bộ semantic assertion qua
sched_2_cpu lặp 5 lần	PASS	Không crash hoặc deadlock quan sát được
os_2_mfq_paging lặp 3 lần	PASS	Không crash hoặc deadlock quan sát được
make report	PASS	LuaLaTeX tạo PDF thành công

```
Running os_0_mfq_paging
Running os_1_mfq_paging
Running os_1_mfq_paging_small_1K
Running os_1_mfq_paging_small_4K
Running os_1_singleCPU_mfq
Running os_1_singleCPU_mfq_paging
Running os_2_mfq_paging
Running os_2_singleCPU_mfq_paging
Running os_sc
Running os_syscall
Running os_syscall_list
Running os_mm64_canonical
Running os_cross_pid_protection
Running os_mem_reuse
Running os_mem_roundtrip
Running os_mem_protection
Running sched
Running sched_0
Running sched_1
Running sched_prio
Running sched_out_of_slot
Running sched_2_cpu
All semantic output assertions passed.
```

Hình 15: Output thực tế của 22 testcase và semantic assertions

Hình 15 cho thấy toàn bộ cấu hình trong `run.sh` đã được thực thi, bao gồm scheduler, paging, syscall, protection và MM64. Dòng kết thúc `All semantic output assertions passed.` quan trọng hơn việc tiến trình chỉ thoát với mã 0, vì nó xác nhận các bất biến đầu ra được mô tả trong `tests/assert/_outputs.sh`.

6.4 Các output thực nghiệm tiêu biểu

Listing 11: Syscall memory làm tăng thống kê page-table walk

```
MM64 pid=1 va=0x0 indexes=[0,0,0,0,0] walks=1 accesses=5 bytes=20480
MM64 pid=1 va=0x0 indexes=[0,0,0,0,0] walks=3 accesses=15 bytes=20480
MM64 pid=1 va=0x0 indexes=[0,0,0,0,0] walks=5 accesses=25 bytes=20480
MM64 pid=1 va=0x0 indexes=[0,0,0,0,0] walks=7 accesses=35 bytes=20480
```

Mỗi memory operation cần một hoặc nhiều PTE access. Output tăng đều theo quan hệ năm access trên một walk.

Listing 12: Sparse branch và round-trip memory

```
MM64 mapped 1 dummy page(s) at 0x200000
MM64 pid=1 va=0x200000 indexes=[0,0,0,1,0] walks=2 accesses=10 bytes=24576
MM64 pid=1 va=0xff00000000000000 indexes=[256,0,0,0,0]
MEM read PID 1 region 3 offset 0 value 65
All semantic output assertions passed.
```

Dung lượng tăng từ 20 480 lên 24 576 byte đúng bằng một bảng PT mới; PGD index 256 chứng minh nửa high-canonical cũng được chấp nhận. Giá trị 65 đọc lại chứng minh chuỗi copy user–kernel–user bảo toàn dữ liệu.

```
./os os_mm64_canonical | grep -E "MM64 mapped|MM64 pid|MM64 rejected"
MM64 mapped 1 dummy page(s) at 0x0
MM64 pid=1 va=0x0 indexes=[0,0,0,0,0] walks=1 accesses=5 bytes=20480
MM64 mapped 1 dummy page(s) at 0x200000
MM64 pid=1 va=0x200000 indexes=[0,0,0,1,0] walks=2 accesses=10 bytes=24576
MM64 mapped 1 dummy page(s) at 0xff00000000000000
MM64 pid=1 va=0xff00000000000000 indexes=[256,0,0,0,0] walks=3 accesses=15 bytes=40960
MM64 rejected non-canonical address 0x1000000000000000
MM64 pid=1 va=0x0 indexes=[0,0,0,0,0] walks=3 accesses=15 bytes=40960
```

Hình 16: Output thực tế của sparse multi-level paging và canonical-address validation

Ảnh terminal thể hiện ba trường hợp tách biệt. Địa chỉ $0x0$ dùng nhánh đầu tiên và cần một walk với năm access; địa chỉ $0x200000$ tạo nhánh PMD index 1 nên dung lượng page table tăng thêm 4096 byte; địa chỉ high-canonical có PGD index 256 vẫn hợp lệ. Ngược lại, địa chỉ $0x1000000000000000$ không thỏa sign extension của bit 56 nên bị từ chối trước khi page-table walk diễn ra.

Listing 13: Các negative test kiểm tra protection

```
Kernel denied PID 1 physical read at 0x0
Kernel denied PID 1 physical write at 0x0
Kernel denied PID 2 physical read at 0x0
Kernel denied PID 2 physical write at 0x0
MM64 rejected non-canonical address 0x1000000000000000
```

Negative test quan trọng vì nó chứng minh hệ thống không chỉ chạy happy path mà còn từ chối input vi phạm. Trong cross-PID test, address 0 là frame hợp lệ của PID 1 nhưng vẫn bị từ chối khi PID 2 truy cập.

6.5 Phân tích output MM64

Khi process kết thúc, `print_pgtbl()` in PID, index của địa chỉ bắt đầu, số walk, số access và số byte page table. Ví dụ:

Listing 14: Định dạng thống kê multi-level paging

```
MM64 pid=1 va=0x0 indexes=[0,0,0,0,0]
walks=7 accesses=35 bytes=20480
```

Quan hệ $\text{accesses} = 5 * \text{walks}$ phù hợp với page-table walk năm cấp. Con số 20480 byte là tổng năm bảng của nhánh đầu tiên; con số tăng theo số bảng con được cấp động. Nếu tính cả lần truy cập dữ liệu vật lý sau walk, một read/write thành công cần thêm một memory access ngoài năm lần đọc bảng.

6.6 Điểm mạnh của testcase hiện tại

- Bao phủ nhiều cấu hình CPU, priority và RAM size.
- Có test bảo vệ âm tính: physical access bị từ chối và địa chỉ non-canonical.
- Có assertion cho branch cấp cao, tái sử dụng data region, cross-PID protection và giá trị đọc lại sau copy user–kernel.

- Gantt diagram giúp đối chiếu scheduler bằng trực quan.
- Test tích hợp chạy scheduler, syscall và memory trong cùng binary.

6.7 Giới hạn của testcase hiện tại

run.sh ghi log vào output/*.output, sau đó tests/assert/_outputs.sh kiểm tra các chuỗi bất biến: syscall table, protection denial, cross-PID ownership, data-region reuse, non-canonical rejection, branch PMD khác 0 và giá trị round-trip. Cách này chịu được thay đổi thứ tự log giữa các CPU.

Các khoảng trống quan trọng:

- chưa có test xác nhận swap-out rồi swap-in giữ nguyên dữ liệu;
- chưa có stress test nhiều CPU và hàng trăm process;
- chưa chạy ThreadSanitizer hoặc AddressSanitizer.

6.8 Đánh giá cơ chế swapping

Code có FIFO victim list, PTE swapped state và hai hàm copy riêng: `__mm_swap_page()` cho RAM sang swap, `__mm_swap_page_in()` cho swap sang RAM. Syscall operation 15 gọi đúng chiều swap-in; PTE swapped không còn mang bit PRESENT. Tuy nhiên khi `ALLOC` không lấy đủ frame, `vm_map_ram()` hiện trả lỗi thay vì chủ động swap victim để hoàn tất allocation. Vì page replacement 64-bit là optional theo đề và phụ thuộc yêu cầu giảng viên, báo cáo chỉ kết luận rằng primitive swap-in/out đã có, còn replacement end-to-end chưa hoàn chỉnh hoặc được kiểm chứng.

7 Các quyết định thiết kế, giới hạn và hướng cải tiến

7.1 Các quyết định thiết kế chính

1. **Một `mm_struct` trên mỗi process:** giữ address space tách biệt, tránh context switch ghi đè một kernel-global `mm`.
2. **PID lookup trong kernel:** tuân thủ yêu cầu không truyền PCB.
3. **FIFO trong từng MLQ bucket:** đơn giản và công bằng giữa cùng priority.
4. **Global locks:** ưu tiên tính đúng đắn và dễ kiểm chứng hơn parallelism tối đa.
5. **Sparse page table cấp động:** chỉ cấp bảng con khi mapping đi qua branch tương ứng, đồng thời giải phóng đệ quy khi process kết thúc.

7.2 Giới hạn kỹ thuật đã xác định

Bảng 19: Giới hạn và tác động

Giới hạn	Tác động	Hướng xử lý
Page replacement chưa có test end-to-end	Có thể còn lỗi khi RAM thật sự cạn	Thêm testcase ép swap và xác nhận dữ liệu
Free-region list không coalescing	Fragmentation metadata	Sắp xếp và gộp vùng kề nhau
Global <code>mmvm_lock</code>	Giảm song song giữa process	Lock theo <code>mm_struct</code>
Build artifacts được track trong Git	Conflict và repository nhiễu	Bỏ <code>obj/*.o</code> , <code>binary</code> và <code>.DS_Store</code>

7.3 Hướng cải tiến

- Thêm testcase end-to-end cho swap-out/swap-in, sau đó bổ sung Clock hoặc LRU.
- Giải phóng sớm branch page-table đã rỗng thay vì chờ process kết thúc.
- Thêm TLB mô phỏng để đo hit/miss và giảm page-table access.
- Viết test harness phân tích output thành event, sau đó kiểm tra scheduler invariants.
- Dùng sanitizer để phát hiện race, use-after-free và memory leak.
- Thu hẹp lock granularity và ghi tài liệu lock ordering để tránh deadlock.

7.4 Đối chiếu kết quả với tiêu chí demonstration

Phần scheduling được chứng minh bằng Gantt diagram và các bất biến multi-CPU. Phần MMU và phân tách user/kernel được chứng minh bằng PID lookup, data-region reuse, copy round-trip và cross-PID denial. Phần multi-level paging được chứng minh bằng năm index, quan hệ năm access trên một walk, mức tăng dung lượng khi tạo branch mới và việc từ chối địa chỉ non-canonical. Ba nhóm bằng chứng này tương ứng trực tiếp với ba thành phần demonstration trong rubric; page replacement end-to-end được ghi nhận riêng như một giới hạn optional thay vì được trình bày như chức năng đã hoàn chỉnh.

8 Kết luận

Bài tập đã tích hợp loader, MLQ scheduler nhiều CPU, timer, syscall, quản lý region, frame và page table trong một simulator thống nhất. Implementation duy trì ranh giới user/kernel bằng PID lookup và frame ownership; hỗ trợ canonical validation, sparse page-table walk năm cấp cùng thống kê access/storage. Kết quả build, 22 testcase và semantic assertions đều thành công, cung cấp bằng chứng cho scheduler, protection, data-region reuse, syscall, sparse paging và round-trip memory.